

Complete Setup & Installation Guide

Document Type	Final Year Project — Installation Guide
Version	1.0.0
Tech Stack	Python 3.12 · FastAPI · React 18 · Docker · SQLite/PostgreSQL
Target OS	Windows 10/11 · macOS 12+ · Ubuntu 20.04+
Prerequisite	Basic command-line familiarity

■ **Tip:** This guide covers EVERY step from zero to running Project Sentinel on your machine. Each section is independent — jump to any section if you already have a tool installed.

Table of Contents

1. System Requirements
2. Installing Python 3.12
3. Installing Node.js 20 (LTS)
4. Installing Git
5. Installing Docker Desktop
6. Project Setup — Backend (FastAPI)
7. Project Setup — Frontend (React)
8. Running the Full Stack
9. Using the Dashboard — Step by Step
10. Optional: API Keys (OpenAI + Shodan)
11. Docker Deployment (One Command)
12. Troubleshooting
13. Project Architecture Reference

1 System Requirements

Component	Minimum	Recommended
CPU	Intel i3 / AMD Ryzen 3	Intel i5/i7 · Ryzen 5/7
RAM	4 GB	8 GB or more
Storage	5 GB free space	20 GB SSD
OS	Windows 10 / macOS 11 / Ubuntu 18.04	Windows 11 / macOS 13 / Ubuntu 22.04
Internet	Required	Stable broadband (for scanning)
Python	3.10+	3.12 (recommended)
Node.js	18+	20 LTS

■ **Warning:** Do not run Sentinel against domains you do not own or have explicit written permission to test. Unauthorized scanning is illegal. Use example.com, your own domain, or a purpose-built lab target for testing.

2

Installing Python 3.12

Required for the FastAPI backend and all AI agents

2.1 Windows

- Open your browser and go to: <https://www.python.org/downloads/>
- Click the yellow "Download Python 3.12.x" button at the top.
- Run the downloaded installer (.exe file).

CRITICAL: On the first installer screen, check the box that says "**Add Python to PATH**" before clicking Install Now.

- Click "Install Now" and wait for completion.
- Verify installation: open Command Prompt (Win+R → type cmd → Enter) and run:

```
python --version # Expected output: Python 3.12.x
```

2.2 macOS

- Option A (Recommended): Install via Homebrew

```
# First install Homebrew if you do not have it: /bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)" # Then install
Python: brew install python@3.12 # Verify: python3 --version
```

- Option B: Download .pkg from <https://www.python.org/downloads/> and run it.

2.3 Ubuntu / Linux

```
sudo apt update && sudo apt upgrade -y sudo apt install -y python3.12 python3.12-venv
python3-pip # Verify: python3.12 --version
```

3

Installing Node.js 20 (LTS)

Required for the React dashboard frontend

3.1 Windows

- Go to: <https://nodejs.org/en/download>
- Click the "LTS" tab and download the Windows Installer (.msi).
- Run the installer and accept all defaults.
- Verify in a NEW Command Prompt window:

```
node --version npm --version # Expected: v20.x.x and 10.x.x
```

3.2 macOS

```
brew install node@20 node --version npm --version
```

3.3 Ubuntu / Linux

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash - sudo apt install -y  
nodejs node --version npm --version
```

■ **Tip:** If you see "npm: command not found" on Linux, run: `sudo apt install -y npm`

4 Installing Git

Version control — also used for project setup

4.1 Windows

- Download from: <https://git-scm.com/download/win>
- Run the installer. Accept all defaults. On the "Adjusting PATH" step, select "Git from the command line and also from 3rd-party software".

```
git --version # Expected: git version 2.x.x
```

4.2 macOS

```
xcode-select --install # installs git automatically # OR via Homebrew: brew install git
```

4.3 Linux

```
sudo apt install -y git git --version
```

5

Installing Docker Desktop

Optional — needed only for the one-command Docker deployment

■ **Note:** Docker is optional. The project runs perfectly without Docker. Use sections 6–8 for the standard local setup.

5.1 Windows & macOS

- Download Docker Desktop from: <https://www.docker.com/products/docker-desktop/>
- Run the installer and follow the setup wizard.
- Start Docker Desktop from your Applications/Start menu.
- Wait for the whale icon in the taskbar/menubar to stop animating (Docker is ready).

```
docker --version docker-compose --version
```

5.2 Linux (Ubuntu)

```
# Install Docker Engine sudo apt update sudo apt install -y docker.io docker-compose
sudo systemctl start docker sudo systemctl enable docker # Add yourself to docker group
(no sudo needed) sudo usermod -aG docker $USER newgrp docker # Verify docker --version
```

Step 1 — Extract the Project

- Extract the provided ZIP file to a location of your choice.
- Example locations:

```
# Windows: C:\Projects\sentinel\ # macOS: ~/Projects/sentinel/ # Linux: ~/sentinel/
```

Step 2 — Open Terminal / Command Prompt

- Windows: Press Win+R, type cmd, press Enter. Or use Windows Terminal.
- macOS: Press Cmd+Space, type Terminal, press Enter.
- Linux: Press Ctrl+Alt+T or search "Terminal".

Step 3 — Navigate to the Backend Directory

```
# Replace the path with where you extracted the ZIP: # Windows: cd  
C:\Projects\sentinel\backend # macOS / Linux: cd ~/Projects/sentinel/backend
```

Step 4 — Create a Virtual Environment (Recommended)

```
# Windows: python -m venv venv venv\Scripts\activate # macOS / Linux: python3.12 -m venv  
venv source venv/bin/activate # You should see (venv) at the start of your prompt
```

Step 5 — Install Python Dependencies

```
pip install -r requirements.txt # This installs: FastAPI, SQLAlchemy, httpx, pydantic,  
uvicorn, openai, etc. # Takes 1-3 minutes on first run.
```

■ **Tip:** If pip is slow, try: `pip install -r requirements.txt -i https://pypi.org/simple/`

Step 6 — Configure Environment Variables

```
# Windows: copy .env.example .env # macOS / Linux: cp .env.example .env # The default  
.env works out of the box with SQLite. # Optionally open .env in Notepad/VSCode to add  
API keys.
```

Step 7 — Start the Backend Server

```
# Make sure you are in the backend/ directory with venv activated uvicorn main:app  
--reload --host 0.0.0.0 --port 8000 # You should see: # INFO: Application startup  
complete. # INFO: Uvicorn running on http://0.0.0.0:8000
```

■ **Tip:** Keep this terminal window open. The backend must stay running while you use the dashboard. Open a NEW terminal window for the frontend setup.

You can verify the backend is running by opening: <http://localhost:8000/docs> in your browser — you should see the Swagger UI API documentation.

7

Project Setup — Frontend (React)

The live dashboard with charts, scan results, and reports

Step 1 — Open a NEW Terminal Window

- Do NOT close the backend terminal. Open a fresh one.

Step 2 — Navigate to the Frontend Directory

```
# Windows: cd C:\Projects\sentinel\frontend # macOS / Linux: cd
~/Projects/sentinel/frontend
```

Step 3 — Install Node.js Dependencies

```
npm install # This installs React, Tailwind CSS, Recharts, Vite, etc. # Takes 1-3
minutes. You will see a node_modules/ folder created.
```

■ **Warning:** If you see "npm: command not found", Node.js is not installed or not in PATH. Go back to Section 3 and install Node.js, then open a FRESH terminal.

Step 4 — Start the Frontend Dev Server

```
npm run dev # You should see: # VITE v5.x.x ready in 500ms # -> Local:
http://localhost:5173/ # -> Network: http://192.168.x.x:5173/
```

■ **Tip:** Open <http://localhost:5173> in your browser. You should see the Project Sentinel dashboard with a dark cybersecurity theme.

8

Running the Full Stack

Summary: both terminals you need open simultaneously

You need TWO terminal windows open at all times:

Terminal	Directory	Command	Port
1 (Backend)	sentinel/backend	uvicorn main:app --reload	8000
2 (Frontend)	sentinel/frontend	npm run dev	5173

Quick-Access URLs

URL	Description
http://localhost:5173	Sentinel Dashboard (main app)
http://localhost:8000	Backend API root
http://localhost:8000/docs	Swagger API documentation (interactive)
http://localhost:8000/health	Health check endpoint

9 Using the Dashboard — Step by Step

Step 1: Open the Dashboard

Navigate to <http://localhost:5173> in Chrome or Firefox. You will see the dark-themed Sentinel dashboard.

Step 2: Start a New Scan

Click "Scans" in the left sidebar. Enter a target domain in the input box (e.g., example.com) and click "Launch Scan". The scan is immediately queued.

Step 3: Watch the Pipeline Run

The scan detail page auto-refreshes every 3 seconds. You will see the status change from PENDING → RUNNING → COMPLETED. The three agents (Scout, Analyst, Oracle) run sequentially in the background.

Step 4: View Discovered Assets

Click the "Assets" tab on the scan detail page. Every subdomain, IP address, open port, and service discovered by the Scout Agent appears here with risk ratings.

Step 5: Review Vulnerabilities

Click the "Vulns" tab. Vulnerabilities are listed by severity (Critical → Low). Click any vulnerability to see the CVE ID, description, CVSS score, and remediation steps.

Step 6: Read the AI Report

Click the "AI Report" tab. The Oracle Agent has generated a full executive summary, technical analysis, threat actor mapping, attack vectors, and a prioritized remediation plan.

Step 7: Cross-Scan Views

Use the "Assets" and "Vulnerabilities" pages in the sidebar to see aggregated data across ALL your scans in one place.

Step 8: Reports Library

The "Reports" page shows all generated reports side-by-side. Click any report to read it in the right-side viewer.

10 Optional: API Keys

OpenAI and Shodan — the app works without these

■ **Important:** Project Sentinel is fully functional WITHOUT any API keys. The built-in threat intelligence database and deterministic report generator provide complete results for demonstrations and academic use.

10.1 OpenAI API Key (Enhances report quality)

- Go to: <https://platform.openai.com/api-keys>
- Create an account and generate a new API key.
- Open backend/.env and set: OPENAI_API_KEY=sk-your-key-here
- Restart the backend. Oracle Agent will now use GPT for polished reports.

10.2 Shodan API Key (Enhances asset discovery)

- Go to: <https://account.shodan.io/register>
- Register for a free account (free tier = 100 queries/month).
- Find your API key at: <https://account.shodan.io>
- Open backend/.env and set: SHODAN_API_KEY=your-key-here
- Restart the backend. Scout Agent will enrich results with Shodan data.

11

Docker Deployment

Run the entire stack with one command

If Docker Desktop is installed (Section 5), you can run the full frontend + backend stack with a single command:

```
# From the root sentinel/ directory: cd sentinel # Build and start all services:  
docker-compose up --build # Wait for both services to start (~2-3 minutes on first run)  
# Then open: http://localhost:3000
```

```
# To stop everything: docker-compose down # To rebuild after code changes:  
docker-compose up --build --force-recreate
```

■ **Tip:** Docker is ideal for demonstrations — it guarantees identical behaviour on any machine. Show your supervisor the docker-compose up output to demonstrate DevOps/cloud deployment knowledge.

12 Troubleshooting

Problem: "python" is not recognized (Windows)

Solution: Python is not in PATH. Uninstall Python and reinstall, making sure to tick "Add Python to PATH" on the first installer screen.

Problem: "pip" is not recognized

Solution: Run: `python -m pip install -r requirements.txt` (use "python -m pip" instead of "pip")

Problem: "ModuleNotFoundError: No module named fastapi"

Solution: The virtual environment is not activated, or requirements were not installed in it. Activate venv first (source `venv/bin/activate`), then re-run `pip install -r requirements.txt`

Problem: Frontend shows "Failed to create scan" error

Solution: The backend is not running. Go to Terminal 1, navigate to `backend/`, and run: `uvicorn main:app --reload`

Problem: "CORS error" in browser console

Solution: The backend CORS settings already allow `localhost:5173` and `localhost:3000`. Make sure you are opening the dashboard via `http://localhost:5173` (not a `file://` URL).

Problem: "npm install" fails with EACCES (macOS/Linux)

Solution: Run: `sudo npm install` OR fix permissions: `sudo chown -R $USER ~/.npm`

Problem: Port 8000 already in use

Solution: Another service is on port 8000. Run the backend on a different port: `uvicorn main:app --reload --port 8001` then update `vite.config.js` proxy to port 8001.

Problem: Scan stays in PENDING forever

Solution: Check the backend terminal for error messages. Common cause: DNS resolution failing for the target domain. Try scanning a well-known domain like `github.com` or `google.com`.

Problem: Docker: "port is already allocated"

Solution: Stop any other services on ports 3000 or 8000 first, then re-run `docker-compose up`.

Problem: SQLite database locked error

Solution: Stop ALL backend instances. Only one uvicorn process should be running at a time.

13 Project Architecture Reference

13.1 Agent Pipeline (Data Flow)

```
[ User: Enter target domain in dashboard ] | v [ FastAPI Backend – POST /api/v1/scans ]
| (Background Task) v +--- SCOUT AGENT -----+ | 1. DNS
subdomain brute-force (50+ subdomains) | | 2. Certificate Transparency (crt.sh) | | 3.
Resolve hostnames to IP addresses | | 4. TCP port scan (18 common ports) | | 5. Shodan
API enrichment (if key set) | +-----+ |
Assets[] v +--- ANALYST AGENT -----+ | 1. Map each asset to
CVE threat database | | 2. HTTP security header checks | | 3. TLS/SSL analysis | | 4.
Compute per-asset risk scores | | 5. Overall 0-100 risk score |
+-----+ | Vulnerabilities[] + RiskScore v
+--- ORACLE AGENT -----+ | 1. OpenAI GPT (if API key) or
built-in engine | | 2. Executive summary (for management) | | 3. Technical analysis (for
security team) | | 4. Threat actor mapping | | 5. Prioritized remediation plan |
+-----+ | Report{} v [ React Dashboard –
live display ]
```

13.2 File Structure

```
sentinel/ ■■■■ backend/ ■ ■■■■ agents/ ■ ■ ■■■■ scout_agent.py # Asset discovery ■ ■
■■■■ analyst_agent.py # CVE correlation ■ ■ ■■■■ oracle_agent.py # AI report generation
■ ■■■■ api/routes.py # REST endpoints ■ ■■■■ models/ # SQLAlchemy DB models ■ ■■■■
db/database.py # Async DB session ■ ■■■■ core/config.py # Environment settings ■ ■■■■
main.py # FastAPI application ■ ■■■■ requirements.txt # Python dependencies ■ ■■■■
.env.example # Environment template ■■■■ frontend/ ■ ■■■■ src/ ■ ■ ■■■■ pages/ #
Dashboard, Scans, Assets, Vulns, Reports, Activity ■ ■ ■■■■ components/ # Reusable
React components ■ ■ ■■■■ utils/ # API client, helper functions ■ ■■■■ package.json #
Node dependencies ■ ■■■■ vite.config.js # Vite + proxy config ■■■■ docker-compose.yml #
One-command deployment ■■■■ README.md
```

13.3 API Endpoints Reference

Method	Endpoint	Description
POST	/api/v1/scans	Create new scan (starts pipeline)
GET	/api/v1/scans	List all scans
GET	/api/v1/scans/{id}	Get scan with assets, vulns, report
DELETE	/api/v1/scans/{id}	Delete a scan
GET	/api/v1/stats	Dashboard statistics
GET	/health	Backend health check
GET	/docs	Swagger interactive API docs

